

AI-Automated Performance Analysis of LLM Inference Frameworks

Automating GPU Profile Trace Analysis with Claude AI
to Continuously Optimize vLLM Performance

Organization	Red Hat
AI Model	Claude Opus 4.6 (Anthropic)
Target Frameworks	vLLM vs SGLang vs TensorRT-LLM
Status	Operational Validated on DeepSeek-R1 NVFP4

1. Introduction

1.1 Background

High-performance LLM inference frameworks such as **vLLM**, **SGLang**, and **TensorRT-LLM** are highly dynamic projects that evolve rapidly to keep pace with the fast introduction of new AI models and use-cases. These frameworks compete to provide the best possible inference performance, requiring intensive work from expert AI engineers and low-level performance specialists.

At **Red Hat**, the focus is on vLLM, with a large engineering effort dedicated to continuously profiling and analyzing vLLM's performance against competing frameworks. This ensures vLLM maintains competitive performance and does not fall behind.

1.2 The Problem

The process of GPU profile analysis is **extremely time-consuming** and requires deep expertise in low-level GPU performance:

- **Trace complexity:** GPU kernel call sequences do not map one-to-one to source code. High-level Python code is broken down into many kernel fragments via compilation phases (e.g., `torch.compile`), and kernel fusion passes merge operations together, hiding information.
- **Expert bottleneck:** A single model profile analysis can take expert programmers days to weeks to fully understand the trace and provide actionable recommendations.
- **Rapid evolution:** Frameworks release optimizations frequently. By the time an expert finishes analyzing one model, the landscape has already changed.

1.3 Goal & Key Insight

The goal is to **use AI to automate the profile analysis process**, removing the human bottleneck entirely. The key insight is that this project is **not about generating new code from scratch** (which is error-prone). Instead, it is about **detecting performance differences** between frameworks and pointing them out. The best optimizations already exist in one framework or another — the AI identifies which techniques from SGLang and TRT-LLM can be adopted to make vLLM faster.

Through extensive experimentation, we proved that with **proper decomposition** of the complex analysis problem into sub-tasks, and **prompt engineering** that embeds expert knowledge and focus areas, it is possible to automate the analysis with high precision using **Claude Opus 4.6** (Anthropic).

1.4 Fully Automated Nightly Flow

The system is designed to run **nightly**, continuously comparing vLLM against SGLang and TensorRT-LLM for key models and use-cases:

- **Auto-Profile Module** executes benchmarks, generates summary tables, and detects problematic cases where vLLM is slower than competing frameworks.
- **Auto-Analyze Module** performs AI-powered Claude profile analysis for each problematic case, detects root causes, and generates a step-by-step improvement plan with source code references.
- **Output:** Performance comparison reports, improvement plans, JIRA tickets with draft PRs. In the morning, an engineer reviews and approves.

Fully Automated Nightly Performance Analysis Flow

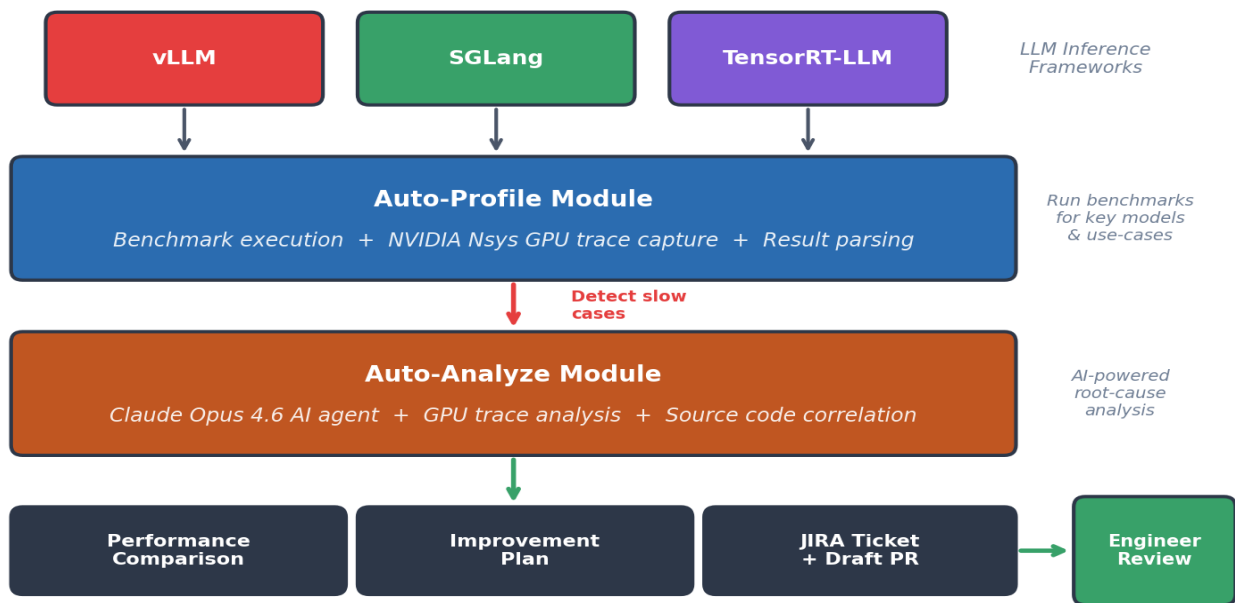


Figure 1: End-to-end automated nightly performance analysis flow

2. Solution Architecture

2.1 System Overview

The system consists of two main modules that work together in a pipeline:

Module	Purpose	Technology
Auto-Profile	Execute benchmarks across vLLM, SGLang, TRT-LLM; capture NVIDIA Nsys GPU traces; parse results	Bash + Docker + NVIDIA Nsys + Claude AI for result parsing
Auto-Analyze	AI-powered analysis: correlate GPU ops to source code, compare frameworks, generate improvement plans	Claude Opus 4.6 (Anthropic) via Claude Agent SDK (Python)

2.2 AI Analysis Pipeline

The core innovation is the **6-step AI analysis pipeline** that decomposes the complex profile analysis problem into manageable sub-tasks. Each step builds on the previous one, progressively deepening the analysis:

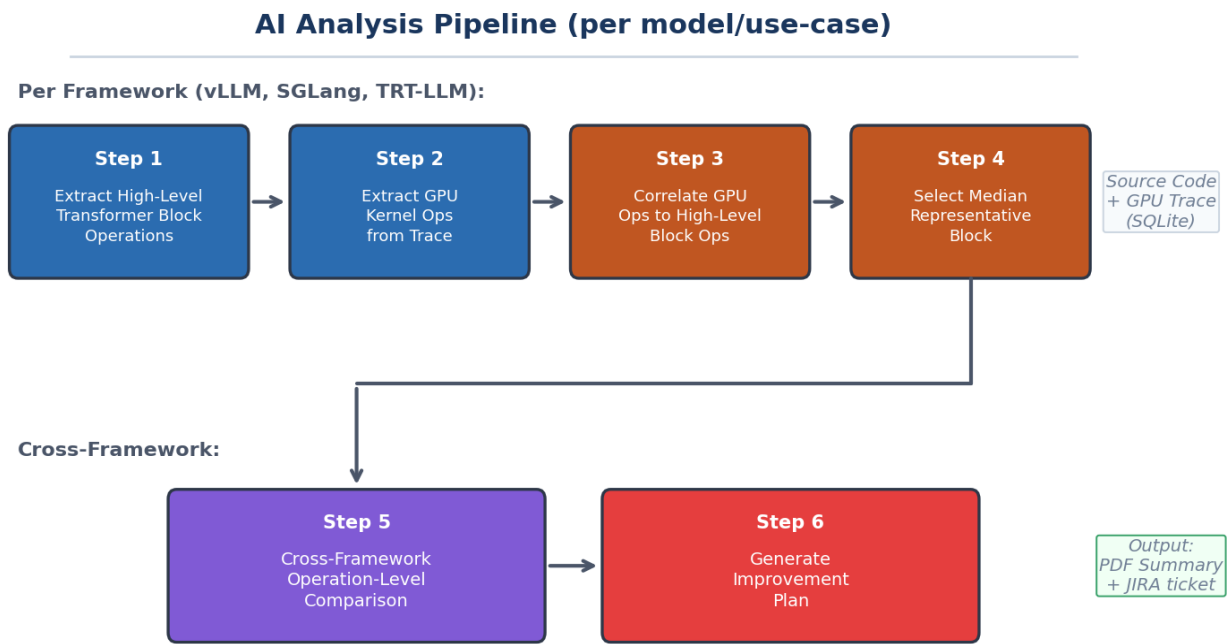


Figure 2: Six-step AI analysis pipeline

Step	Task	Input	Output
1	Extract high-level transformer block operations from source code	Framework source code	Ordered list of operations per block type (Dense vs MoE) with source references
2	Extract GPU kernel operations from NVIDIA Nsys trace	SQLite trace file (GPU 0)	Top 1000 GPU ops sorted by start time with kernel names, durations, streams
3	Correlate GPU ops to high-level transformer block operations	Steps 1 + 2 outputs	Per-block tables with GPU op to high-level mapping, timing summaries, observations
4	Select median representative transformer block	Step 3 output	Single block (median wall time) with complete operation listing

5	Cross-framework operation-level performance comparison	Step 4 outputs (all 3 frameworks)	Matched operation table with durations, kernel differences, critical path analysis
6	Generate prioritized improvement plan	Step 5 output + source code	Ordered plans with root cause analysis, source code references, implementation steps

Table 1: AI analysis pipeline steps

2.3 Key Design Decisions

- **Task decomposition over monolithic prompting:** Breaking the analysis into 6 sequential steps prevents the AI from being overwhelmed by trace complexity. Each step has a focused objective with clear input/output contracts.
- **Expert knowledge embedding:** Prompts encode domain expertise (e.g., "kernel fusion passes hide source code mapping," "look for torch.compile-generated Triton kernels," "MoE blocks use dual CUDA streams"). This guides the AI to focus on relevant patterns.
- **Comparison-based approach:** Rather than inventing optimizations, the AI compares actual GPU kernel implementations across frameworks. The best optimization already exists in one framework — the task is to find and transfer it.
- **Claude Opus 4.6 with tools:** The AI agent has access to Read, Write, and Bash tools, allowing it to navigate codebases, query SQLite traces, and produce structured output files.

3. Example: DeepSeek-R1 NVFP4 on B200

This section illustrates the system output using a real analysis of **DeepSeek-R1 NVFP4** (671B parameters, FP4 quantized) on NVIDIA B200 GPUs.

3.1 Test Configuration

Parameter	Value
Model	nvidia/DeepSeek-R1-NVFP4 (DeepseekV3ForCausalLM)
Architecture	61 layers: 3 Dense + 58 MoE MLA Attention 256 routed + 1 shared expert
Precision	NVFP4 (4-bit weights, BF16 activations)
GPU	NVIDIA B200 (Blackwell), TP=4
Workload	Decode phase, batch=1, ISL=4, OSL=1024
Profiling	NVIDIA Nsys with CUDA + NVTX tracing

Table 2: DeepSeek-R1 analysis configuration

3.2 Transformer Block Structure (Step 1 Output)

The AI extracts the high-level operation sequence for MoE transformer blocks from vLLM source code. DeepSeek-R1 has two block types: Dense (layers 0-2) and MoE (layers 3-60). The table below shows the MoE block structure with 20 high-level operations:

#	Operation	Source	Execution Mode
1	Input RMSNorm + Residual Add	layernorm.py:93	Fused Add + RMSNorm kernel
2	Fused QKV-A Projection [7168]->[2112]	deepseek_v2.py:759	NVFP4 CUTLASS GEMM (splitK)
3	Q-A LayerNorm [1536]	deepseek_v2.py:777	RMSNorm (Triton)
4	Q-B Projection [1536]->[6144]	deepseek_v2.py:778	NVFP4 CUTLASS GEMM (TP-sharded)
5-6	KV-A LayerNorm + Split	mha.py:149-150	RMSNorm on kv_c [512]

7	RoPE (Rotary Embedding)	deepseek_v2.py:816	Applied to q_pe and k_pe
8	KV Cache Write	mha_attention.py:524	concat_and_cache_mha kernel
9	Q-Nope Absorption (BMM)	mha_attention.py:602	BF16 BMM decode path
10	Concat Q for decode	mha_attention.py:624	FlashInfer MLA format
11	Decode MLA Attention (FMHA)	mha_attention.py:635	FLASHINFER_MLA decode kernel
12	V Up-Projection (BMM)	mha_attention.py:787	BF16 BMM latent->v_dim
13	Output Projection + AllReduce	deepseek_v2.py:801	NVFP4 GEMM + TP AllReduce
14	Post-Attn RMSNorm + Residual	deepseek_v2.py:1019	Fused Add + RMSNorm
15	Gate Linear (Router) [7168]->[256]	deepseek_v2.py:251	BF16 GEMM (replicated)
16	MoE Routed Experts (top-8)	flashinfer_fp4_moe.py:260	FP4 GEMM1+SwiGLU+GEMM2
17	Shared Expert MLP (parallel)	deepseek_v2.py:216	Separate CUDA stream
18-20	Scale + Add Shared + AllReduce	deepseek_v2.py:356-375	Combine + TP/EP reduce

Table 3: High-level operations in a MoE transformer block (vLLM)

3.3 GPU Ops to Block Correlation (Step 3 Output)

The AI correlates 1000 GPU kernel operations from the Nsys trace to transformer blocks. It identifies **3 Dense blocks** (21 GPU ops each) and **28 MoE blocks** (33 GPU ops each, spanning 2 CUDA streams). The table below shows timing for each detected block:

Block	GPU Ops	Wall Time	Kernel Sum	Stream 19	Stream 392
Pre-block (Embedding)	0-9	7.83 ms	132.24 us	132.24 us	0 ns
Layer 0 Dense	10-30	184.78 us	184.21 us	184.21 us	0 ns
Layer 1 Dense	31-51	111.66 us	111.31 us	111.31 us	0 ns
Layer 2 Dense	52-72	109.70 us	108.91 us	108.91 us	0 ns
Layer 3 MoE	73-105	121.90 us	153.10 us	132.43 us	20.67 us
Layers 4-26 MoE	106-864	~122-126 us	~153-158 us	~132-137 us	~20-22 us
Layer 27 MoE (median)	865-897	123.34 us	155.31 us	133.58 us	21.73 us
Layers 28-30 MoE	898-996	~122-124 us	~154 us	~133 us	~20-21 us

Table 4: Block detection and timing summary (vLLM). Kernel sum > wall time indicates stream parallelism.

Key observation: MoE blocks use two CUDA streams (main stream 19 for routed experts, stream 392 for shared expert). The kernel time sum exceeds wall time, indicating effective stream overlap of ~32 us per block.

3.4 Median Block Detail (Steps 3-4 Output)

Layer 27 is selected as the median block (wall time 123.34 us, rank 15/28). Below is the complete GPU kernel sequence with high-level operation correlation:

Op#	High-Level Operation	GPU Kernel	Duration	Stream
865	Input RMSNorm (+residual)	triton_red_fused_*_rsqrt	4,801 ns	s19
866	QKV-A Proj (splitK GEMM)	nvjet_tst_*_splitK_TNT	8,000 ns	s19
867	QKV-A Proj (splitK reduce)	splitKreduce_kernel	1,984 ns	s19
868	KV-A LayerNorm	triton_poi_fused_3	1,696 ns	s19
869	Q-A LayerNorm	triton_red_fused_4	2,081 ns	s19
870	Q-B Projection	nvjet_tst_*_TNT	5,600 ns	s19
871	RoPE	triton_poi_fused_*_mul	1,952 ns	s19

872	KV Cache Write	concat_and_cache_mla	2,208 ns	s19
873	Q-Nope Absorption BMM	nvjet_tst_*_NNT	3,457 ns	s19
874	Concat Q for decode	CatArrayBatchedCopy	2,240 ns	s19
875	FMHA Decode Attention	fmhaSm100fKernel	15,169 ns	s19
876	V Up-Projection BMM	nvjet_tst_*_TNN	3,905 ns	s19
877	Output Projection	nvjet_tst_*_TNT	11,745 ns	s19
878	Output AllReduce	cross_device_reduce_1stage	6,880 ns	s19
879	Post-Attn RMSNorm	triton_red_fused_*_moe	4,320 ns	s19
880	MoE Clone (memcpy)	Memcpy_DtoD_14336B	1,313 ns	s19
881	Gate Linear (splitK)	nvjet_tst_*_splitK_TNN	4,544 ns	s19
882	Shared: gate_up quant	cvt_fp16_to_fp4	3,424 ns	s392
883	Shared: gate_up GEMM	device_kernel (Cutlass)	9,473 ns	s392
884	Gate Linear (reduce)	splitKreduce_kernel	2,656 ns	s19
...	... (routing, MoE GEMMs, finalize)	...	...	...
897	MoE AllReduce (TP/EP)	cross_device_reduce_1stage	7,456 ns	s19

Table 5: Median block (Layer 27 MoE) GPU kernel correlation. 33 total ops, 28 on stream 19, 5 on stream 392.

3.5 Cross-Framework Comparison (Step 5 Output)

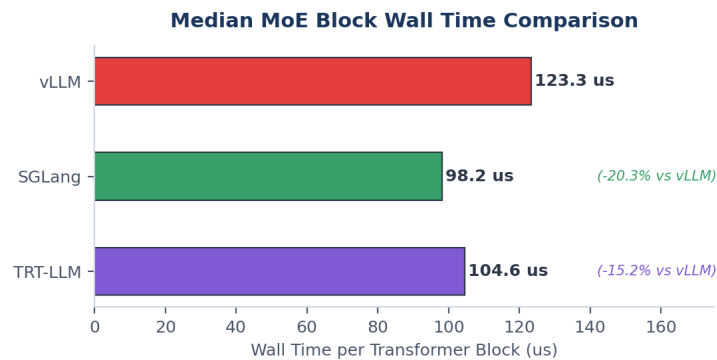


Figure 3: Median MoE block wall time comparison

Metric	vLLM	SGLang	TRT-LLM
Median Block Layer	27	54	12
Wall Time	123.34 us	98.25 us	104.55 us
Kernel Sum	155.31 us	144.52 us	154.32 us
GPU Kernel Count	33	26	24
Streams Used	s19 + s392	s539 + s540	s32 + s5263
Ranking	3rd (slowest)	1st (fastest)	2nd

Table 6: Cross-framework comparison overview

3.6 Key Performance Differences

#	Difference	vLLM	SGLang	TRT-LLM	Key Insight
1	AllReduce + RMSNorm	11.2K ns (2 kernels)	8.6K ns (1 fused)	7.4K ns (1 fused)	vLLM lacks Lamport protocol fusion
2	FMHA Decode	15.2K ns	15.4K ns	10.8K ns	TRT 30% faster (PDL pipelining)
3	MoE Gate Overhead	24.6K ns (8 kernels)	9.5K ns (overlapped)	19.7K ns	vLLM has 5.4K ns extra type casts
4	MoE GEMM Tile Config	19.3K ns t128x8x512	19.5K ns t128x8x512	22.1K ns t128x16x256	TRT suboptimal tile for batch=1
5	Shared Expert GEMM	9.5K ns (Cutlass)	8.9K ns (cuDNN)	4.6K ns (CUDA Core)	CUDA Core FP4 2x faster at M=1

Table 7: Top performance differences detected by AI analysis (duration in nanoseconds)

3.7 Critical Path Analysis

The AI performs critical path analysis to understand why SGLang achieves the lowest wall time despite similar kernel-level operations:

- **SGLang's strategy:** Fuses AllReduce+RMSNorm (Lamport protocol), runs routed experts on ALT stream (hiding 46K ns latency), keeps shorter shared expert (18.8K ns) on critical path, and starts router on ALT stream during post-attention processing.
- **TRT-LLM's strategy:** Uses PDL pipelining to overlap block boundaries (~20K ns savings), fuses RoPE+KV Cache+Concat on ALT stream, and applies aggressive 5-way POST_MOE fusion.
- **vLLM's gaps:** Fully serial execution on main stream (no parallelism optimization), separate AllReduce+RMSNorm kernels, extra type cast overhead in MoE gate, and 33 kernel launches vs 24-26 for competitors.

3.8 AI-Generated Improvement Plans (Step 6 Output)

The AI generates 12 prioritized improvement plans with detailed root cause analysis, source code references, and step-by-step implementation instructions. The table below summarizes the plans ordered by estimated impact:

#	Plan	Target	Impact (ns/block)	Difficulty
1	Fuse AllReduce+Residual+RMSNorm (Lamport protocol, 2 fusion points)	vLLM	~7,600	Medium
2	Eliminate MoE Gate/Router overhead (replace splitK, remove casts/memcpy)	vLLM	~5,400	Medium
3	Improve FMHA Decode performance (PDL pipelining, FMHA config tuning)	vLLM/SGL	~4,500 each	Hard
4	Optimize dual-stream MoE strategy (routed on ALT, shared on MAIN)	vLLM/TRT	~3-8K each	Medium
5	Optimize QKV-A Projection (splitK GEMM for batch=1 decode)	TRT-LLM	~7-10K	Medium
6	Optimize Q-A LayerNorm kernel (Triton kernel for small tensors)	TRT/SGL	~5.8K/1.8K	Easy
7	Adopt CUDA Core FP4 GEMM for shared expert (M=1 optimization)	vLLM/SGL	~4,500 each	Medium
8	Optimize MoE GEMM tile config (t128x8x512 for batch=1)	TRT-LLM	~2,900	Easy
9	Fuse POST_MOE operations (finalize + add + AllReduce)	vLLM	~2,700	Medium

10	Add attention prep parallelism (RoPE+KV on ALT stream)	vLLM/SGL	~2-3K each	Medium
11	Enable PDL pipelining (inter-block kernel overlap)	vLLM/SGL	~5-20K	Hard
12	Reduce kernel launch count (automatic via Plans 1+2+9)	vLLM	~2,700	Auto

Table 8: AI-generated improvement plans, ordered by priority. Shaded rows are critical priority.

Projected vLLM savings: If all applicable plans are implemented, the estimated total savings for vLLM is **~20K ns per block**, reducing wall time from 123.3 us to ~103 us per MoE block (**~16% improvement**). This would bring vLLM to parity with SGLang (98.2 us) and TRT-LLM (104.6 us). The largest contributors are Plan 1 (AllReduce+RMSNorm fusion, ~7.6K ns), Plan 2 (MoE gate overhead, ~5.4K ns), and Plan 3 (FMHA decode, ~4.5K ns).

3.9 Example Plan Detail: Fuse AllReduce+RMSNorm

Each plan includes detailed root cause analysis and implementation steps. Below is a summary of Plan 1 (highest priority for vLLM):

Problem: vLLM uses 2 separate kernels for post-attention processing:

- 1. cross_device_reduce_1stage (AllReduce): 6,880 ns
- 2. triton_red_fused (RMSNorm + residual): 4,320 ns
- Total: 11,200 ns vs SGLang's fused 8,641 ns and TRT-LLM's fused 7,360 ns

Root cause: vLLM's AllReduce (in RowParallelLinear) and RMSNorm (in DeepseekV2DecoderLayer) are decoupled. SGLang/TRT-LLM use a single Lamport-protocol kernel that fuses both operations.

AI-generated implementation steps:

1. Import Lamport fused AllReduce+Residual+RMSNorm kernel (from flashinfer/sgl-kernel)
2. Add forward_with_allreduce_fusion() method to vLLM's RMSNorm class
3. Modify RowParallelLinear to optionally skip internal AllReduce
4. Modify DeepseekV2DecoderLayer.forward() to defer AllReduce to fused RMSNorm
5. Apply same pattern for both post-attention and post-MoE (2 fusion points)
6. Guard with config flag + CUDA graph mode check; test for correctness

Metric	Value
Savings per fusion point	~3,800 ns
Fusion points per block	2 (post-attention + post-MoE)
Total savings per block	~7,600 ns
Source files to modify	vllm/model_executor/layers/layernorm.py vllm/model_executor/models/deepseek_v2.py vllm/distributed/fused_allreduce.py (new)

4. Conclusion & Impact

4.1 Results Summary

The AI-automated analysis system has been validated on DeepSeek-R1 NVFP4, producing results consistent with expert-level analysis:

Metric	Value
Model analyzed	DeepSeek-R1 NVFP4 (671B params) on NVIDIA B200 x4 (TP=4)
GPU operations correlated	1000 ops mapped to 31 complete transformer blocks
Performance gaps detected	10+ distinct differences across 12 execution phases
Improvement plans generated	12 prioritized plans with source code references
Projected vLLM improvement	~20% wall time reduction (123 us -> ~98-103 us per block)
Analysis time	Fully automated (vs days/weeks of expert manual analysis)

Table 9: Analysis results summary

4.2 Key Achievements

- **Automated what was previously impossible to scale:** GPU profile trace analysis that took expert programmers days to weeks is now fully automated, running nightly.
- **High-precision AI analysis:** By decomposing the problem into 6 sequential steps and embedding expert knowledge into prompts, Claude Opus 4.6 produces analysis that matches expert-level quality with detailed source code references.
- **Actionable output:** The system does not just identify problems — it generates step-by-step implementation plans with specific source code locations, kernel names, and expected performance impact.
- **Comparison-based approach:** By comparing frameworks rather than inventing optimizations, the AI leverages the collective innovation of all three framework teams. The best optimization already exists somewhere — the AI finds and transfers it.

4.3 Business Impact

Impact Area	Before	After
Analysis cycle time	Days to weeks per model	Nightly automated runs
Expert programmer time	80%+ on trace analysis	Review and approve only
Coverage	1-2 models analyzed per quarter	All key models analyzed nightly
Time to detect regressions	Weeks (if detected at all)	Next morning
vLLM competitiveness	Reactive (after complaints)	Proactive (continuous calibration)

Table 10: Business impact comparison

4.4 Vision

This system transforms vLLM performance optimization from a reactive, expert-bottlenecked process into a **continuous, automated calibration loop**. Every night, the system benchmarks, profiles, analyzes, and generates improvement plans. Every morning, engineers review JIRA tickets and approve draft PRs. The result is that vLLM continuously absorbs the best optimizations from competing frameworks, ensuring it is always competitive and never falls behind — without being bottlenecked on the availability of scarce low-level GPU performance experts.